

Rappels cryptographiques

Rida Khatoun

Télécom-Paris

rida.khatoun@telecom-paris.fr

- Services de base de la cryptographie
- Rappels sur les algorithmes symétriques
- Méthodes de partage de clefs
- Rappels sur les algorithmes asymétriques
 - RSA
 - Elgamal
 - Paillier
 - Rabin
- Fonctions de hachage

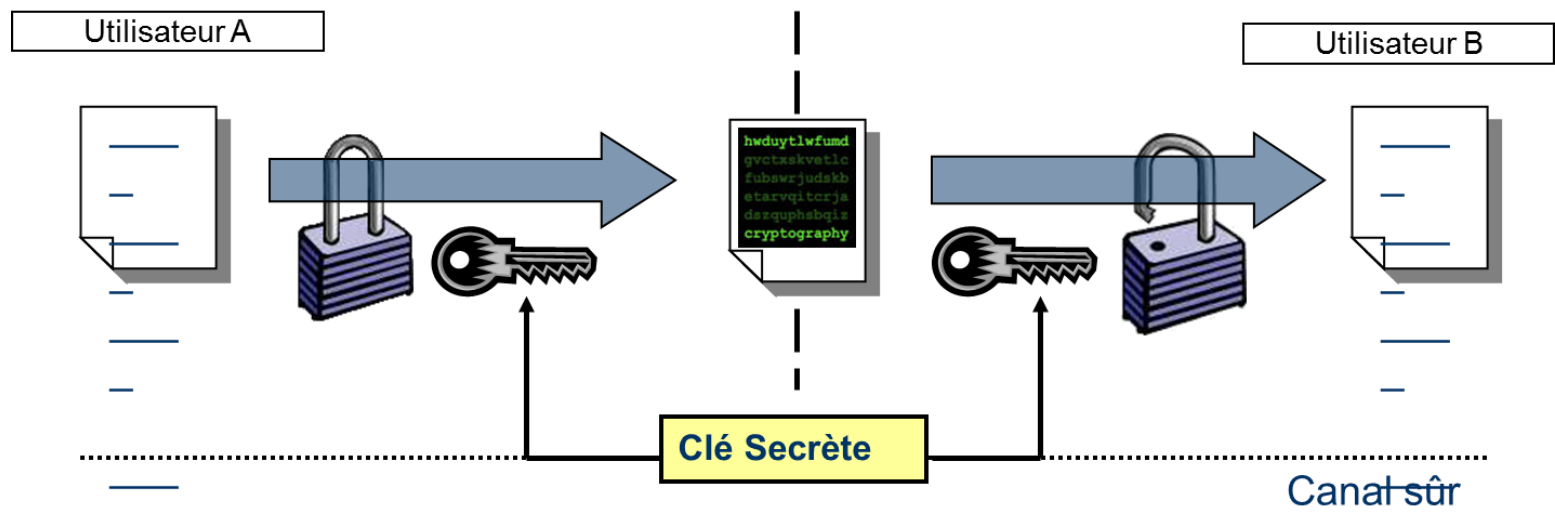
Cryptographie

- **Le Chiffrement :**
 - Transformer une information claire en information chiffrée, appelé cryptogramme.
- **La Cryptographie :**
 - Science qui étudie les procédés de chiffrement et de déchiffrement.
- **La Cryptanalyse :**
 - Etude des procédés de décryptage et de la sécurité des algorithmes de chiffrement.
- **La Cryptologie :**
 - Cryptographie + Cryptanalyse.

- **L'Authentification :**
 - Vérification de l'identité d'un objet, d'une personne.
- **L'Intégrité d'un objet : (document, fichier, message ...)**
 - Garantie que cet objet n'a pas été modifié.
- **La Confidentialité :**
 - L'assurance qu'un document ne sera pas lu par un tiers qui n'en a pas le droit.
- **La Non-répudiation :**
 - L'émetteur d'un message ne peut pas nier l'avoir envoyé et le récepteur l'avoir reçu.

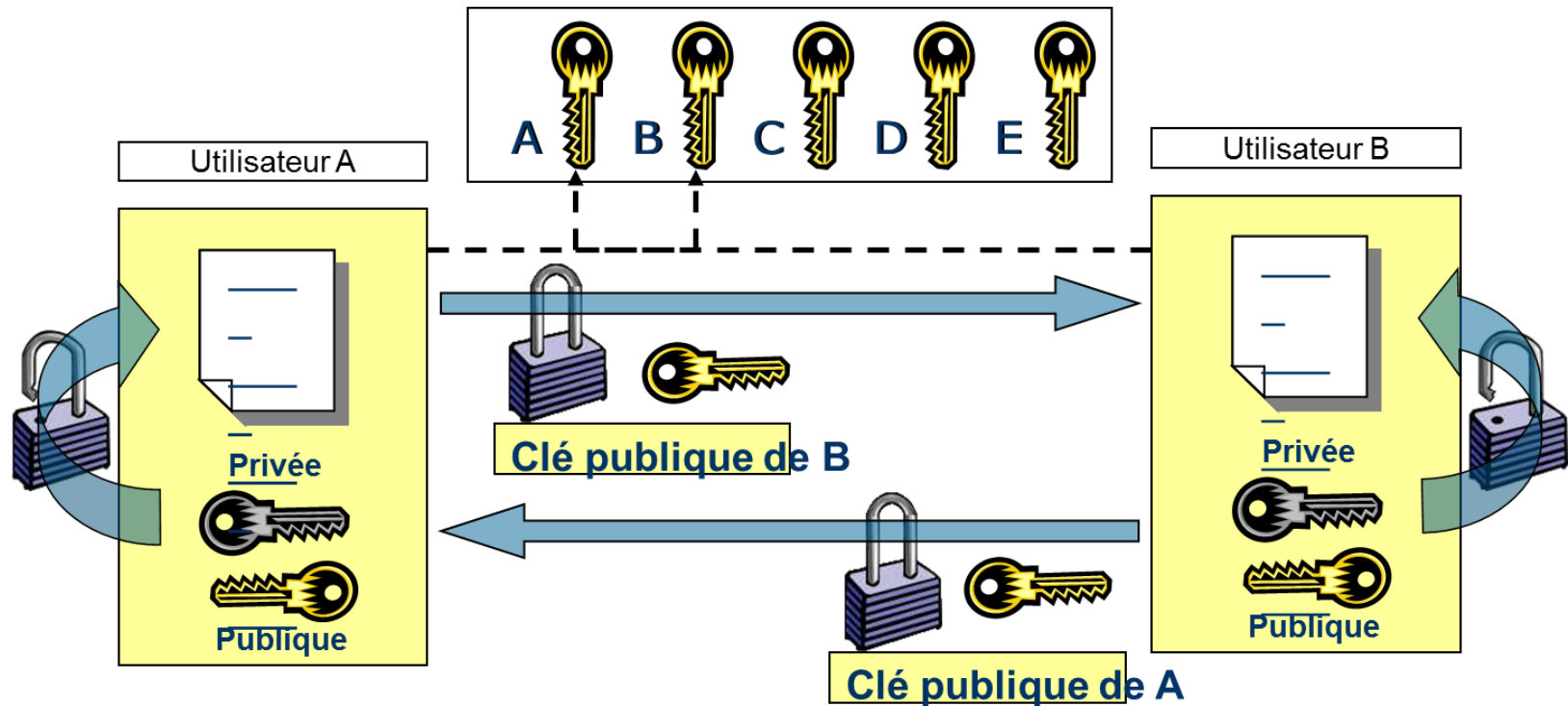
- Cryptographie symétrique

- Algorithmes : AES, 3-DES, CAMELIA, RC4,
- $n(n-1)/2$ clés à gérer pour n personnes
- Problème de transmission de la clé secrète par un canal sûr



- Cryptographie Asymétrique

- Algorithmes : RSA, El Gamal, Paillier ...
- Beaucoup plus lent que le chiffrement symétrique



- Dans une architecture à clés publiques :
 - Les clés privées doivent être générées et stockées de manière sûre
 - Les clés publiques doivent être diffusées le plus largement possible.
- Lorsqu'on utilise la clé d'un correspondant, il est nécessaire de s'assurer :
 - qu'elle appartient bien à ce correspondant
 - que l'émetteur de cette clé est digne de confiance
 - qu'elle est toujours valide

Chiffrement symétrique vs Chiffrement asymétrique

Chiffrement symétrique

Avantages

- Rapidité des opérations (adapté à du trafic en temps réel) ;
- Clés courtes (256 bits suffisent actuellement) ;

Inconvénients

- Difficulté d'échange sécurisé des clés secrètes : comment le faire en protégeant ce secret ?

Chiffrement asymétrique

- Facilité d'échange des clés : les seules clés qui ont besoin d'être échangées sont des clés publiques (dont il faut assurer la protection en intégrité) ;

- Lenteur des opérations (peu adapté à du trafic en temps réel) ;
- Grande taille des clés (2048 bits minimum actuellement) ;

Exemples d'algorithmes sûrs (2019)

- AES, Chacha20,

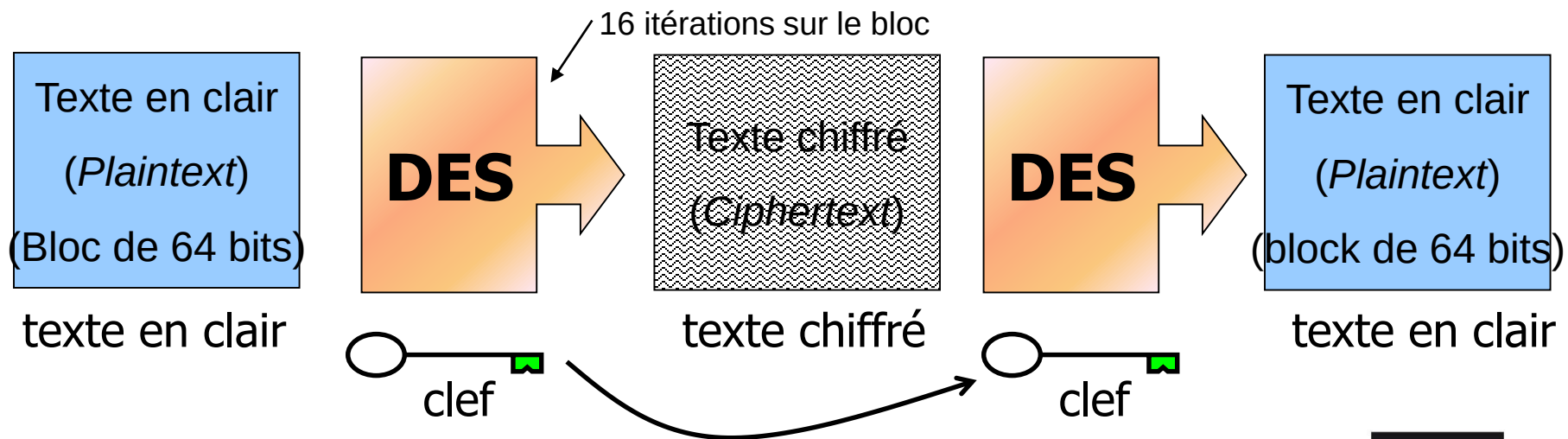
- RSA, El Gamal

Algorithmes de chiffrement symétrique

Chiffrement à clef symétrique : DES

- Data Encryption Standard (DES)
 - Adopté par le NIST* en 1977
 - Chiffrement par bloc de 64 bits (CBC)
 - Clef de 56 bits
 - Largement répandu pour des applications financières

*National Institute of Standards and Technology : agence du Département du Commerce aux États-Unis



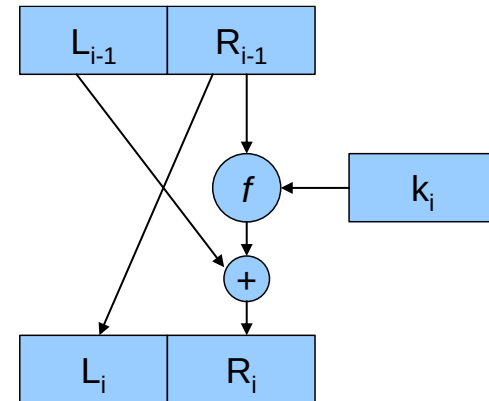
Chiffrement à clef symétrique : DES

- DES se déroule en trois étapes

x est un bloc de 64 bits

- Permutation initiale (IP) appliquée à x pour obtenir une chaîne $IP(x) = L_0R_0$
- On effectue 16 tours de chiffrement (rondes)

$$\begin{cases} L_i = R_{i-1} \\ R_i = L_{i-1} (+) f(R_{i-1}, K_i) \end{cases}$$



- On effectue une permutation inverse IP^{-1} à $R_{16}L_{16}$

Chiffrement à clef symétrique : DES

- Diversification de la clef dans DES :
 - K de 64 bits est réordonné dans PC-1 qui supprime les bits de parités (en 8, 16,...,64)
 - LS_i : rotation circulaire vers la gauche d'une ou deux positions selon la valeur de i
 - PC2 : autre permutation de bits

Table 3.1 Permuted choice 1 (PC-1)

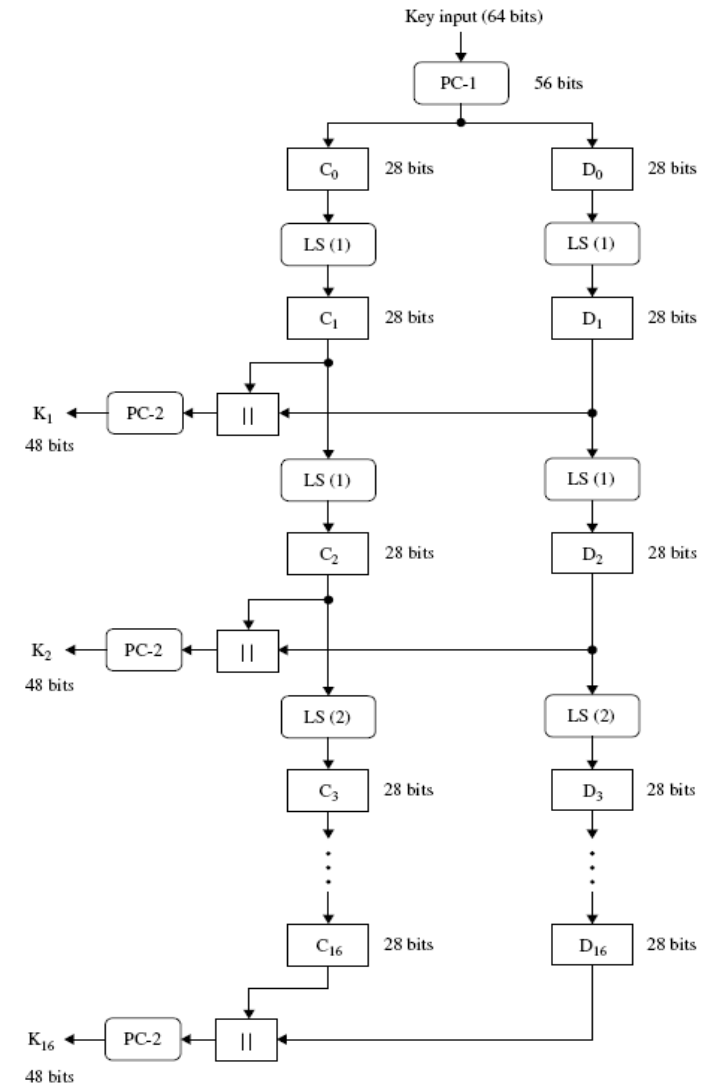
57	49	41	33	25	17	9	1	58	50	42	34	26	18
10	2	59	51	43	35	27	19	11	3	60	52	44	36
63	55	47	39	31	23	15	7	62	54	46	38	30	22
14	6	61	53	45	37	29	21	13	5	28	20	12	4

Table 3.2 Schedule for key shifts

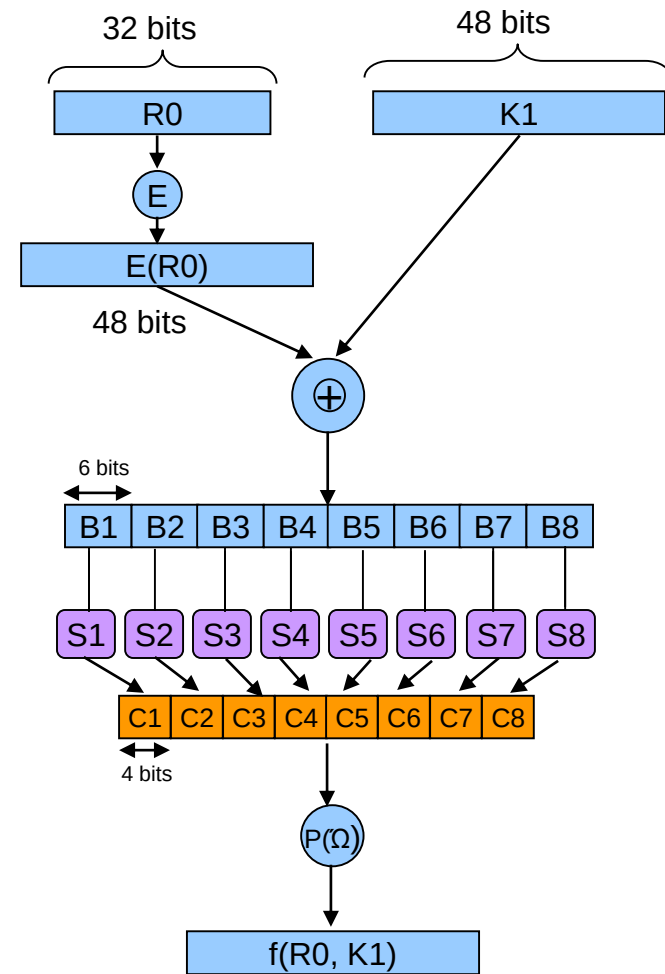
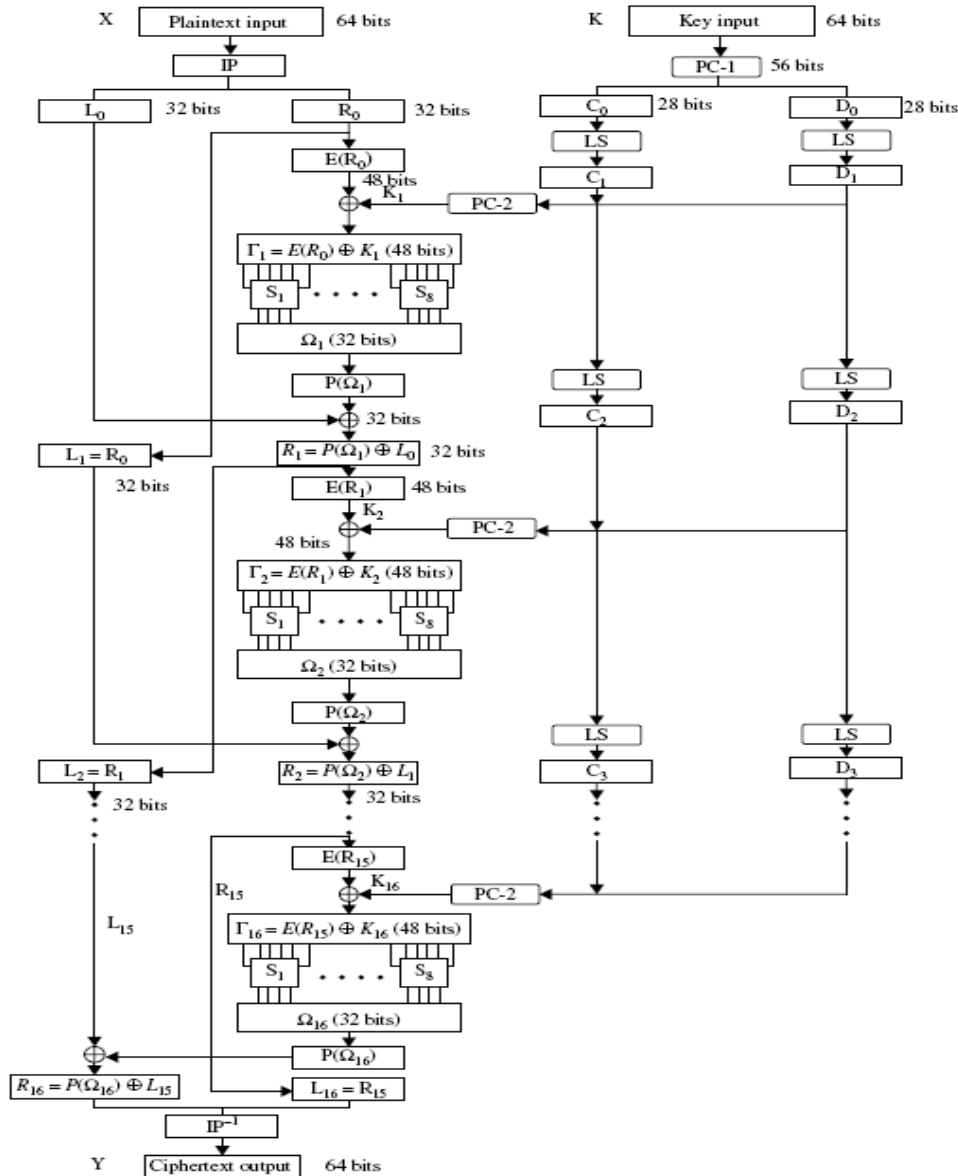
Round number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Number of left shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Table 3.3 Permuted choice 2 (PC-2)

14	17	11	24	1	5	3	28	15	6	21	10
23	19	12	4	26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40	51	45	33	48
44	49	39	56	34	53	46	42	50	36	29	32



Chiffrement à clef symétrique : DES



Chiffrement à clef symétrique : DES

Inverse of initial permutation, IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Initial permutation (IP)

L_i	58	50	42	34	26	18	10	2
	60	52	44	36	28	20	12	4
	62	54	46	38	30	22	14	6
	64	56	48	40	32	24	16	8
R_i	57	49	41	33	25	17	9	1
	59	51	43	35	27	19	11	3
	61	53	45	37	29	21	13	5
	63	55	47	39	31	23	15	7

Permutation function P

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

E bit-selection table

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Chiffrement à clef symétrique : DES

- Boîte de substitution S_i
- Ex: si $B5 = 101110$
alors, $S_5^{10}(0111)$
 $\Rightarrow C5 = 8 \text{ hex} = (1000)_b$

		S-boxes															
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S_1	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Chiffrement à clef symétrique : DES

- **Avantages et applications de DES**
 - Facile à comprendre et implémenter
 - Sécurité indépendante de l'algorithme lui-même
 - Rapide
 - Clé relativement petite
 - Résultat statistiquement « plat » après 16 tours : espaces et fréquences indétectables
 - Légère modification du texte provoque changements importants dans le *ciphertext*
- **Applications actuelles**
 - Chiffrer les paiements par carte de crédits (*UEPS*)
 - Protocoles d'authentification comme *Kerberos*

Chiffrement à clef symétrique : AES

- AES (Advanced Encryption Standard)
 - Histoire
 - Inventé par Joan Daemen et Vincent Rijmen
 - Adopté par NIST* en 2000
 - Successeur de DES
 - Standard de chiffrement pour les organisations du gouvernement des Etats-Unis
 - Chiffrement : symétrique (CBC)
 - Bloc : 128 bits organisés sous forme matricielle
 - Clefs : taille 128, 192, 256 bits

* National Institute of Standards and Technology

Chiffrement à clef symétrique : AES

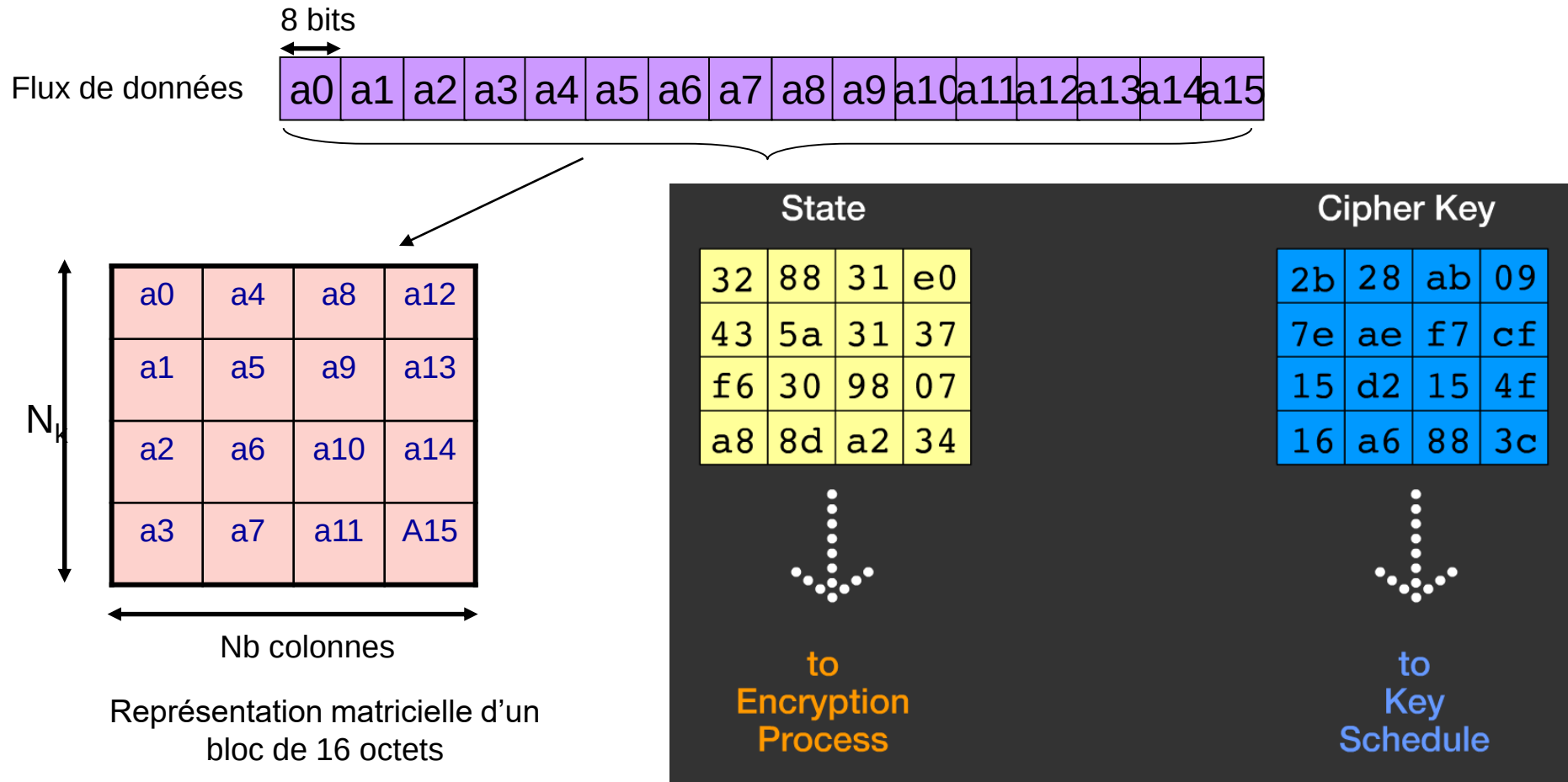
- Propriétés d'AES

- Plusieurs longueurs de clef et de bloc sont possibles : 128, 192, ou 256 bits
- Nombre de cycles ("rondes") varie en fonction de la longueur des blocs et des clés (de 10 à 14)
- Structure générale ne comprend qu'une série de transformations/permutations/sélections
- Beaucoup plus performant que le DES
- Parallélisme peut être implémenté

Chiffrement à clef symétrique : AES

- Principe de l'algorithme :
 - Chiffrement AES consiste en
 - Une addition initiale de clef (*AddRoundKey*)
 - $N_r - 1$ tours (rondes) chacun divisé en 4 étapes
 - Quatre étapes d'une ronde (tours)
 - *SubBytes* : substitution non-linéaire où chaque octet est remplacé par un autre choisi d'une table SBox
 - *ShiftRows* : transposition où chaque élément de la matrice est décalé cycliquement à gauche d'un certain nombre de colonnes
 - *MixColumns* : effectue un produit matriciel
 - *AddRoundKey* : addition de chaque octet avec l'octet correspondant dans une clé de tour obtenue par diversification

Chiffrement à clef symétrique : AES



hexadecimal notation:

Ex: 32 = 00110010 (1 byte)
3hex 2hex

Chiffrement à clef symétrique : AES

Flux de données

Ex : $32 \text{ xor } 2b = 19$ en hex

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

32	88	31	e0
43	5a	31	37
f6	30	98	07
a8	8d	a2	34

AddRoundKey

Cipher Key			
2b	28	ab	09
7e	ae	f7	cf
15	d2	15	4f
16	a6	88	3c

initial
round

1-SubBytes

2-ShiftRows

3-MixColumns

4-AddRoundKey

Round key 0

9
rounds

SubBytes

ShiftRows

AddRoundKey

Round key 10

final
round

Chiffrement à clef symétrique : AES

- SubBytes (State)

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

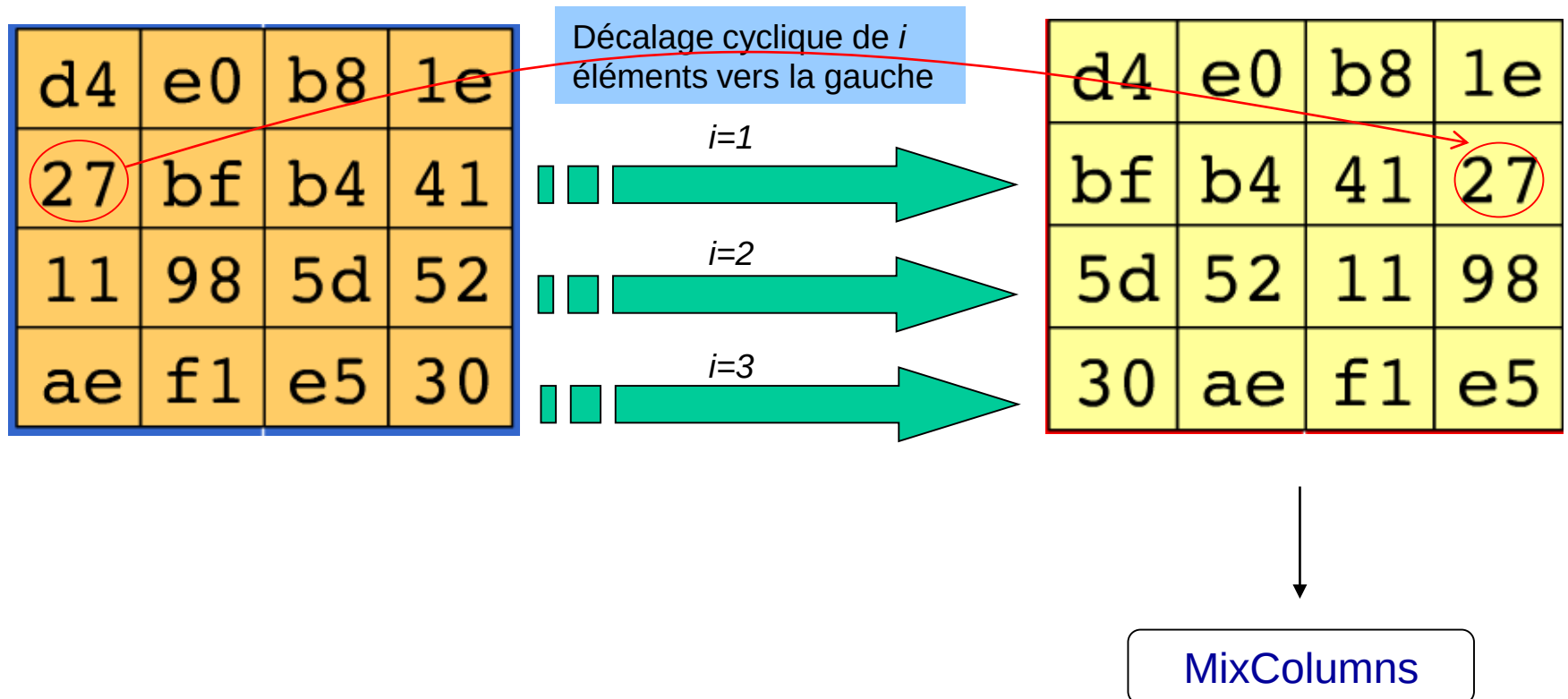
d4	e0	b8	1e
27	bf	b4	41
11	98	5d	52
ae	f1	e5	30

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	e9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	db	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

AES SBox

Chiffrement à clef symétrique : AES

- ShiftRows (State)

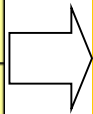


Chiffrement à clef symétrique : AES

- MixColumns (State)

multiplication de polynômes modulo un polynôme irréductible.

d4	e0	b8	1e
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5



e0	b8	1e
b4	41	27
52	11	98
ae	f1	e5

02	03	01	01
01	02	03	01
01	01	02	03
03	01	01	02

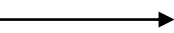
*

d4
bf
5d
30

=

04
66
81
e5

MixColumns



04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

Chiffrement à clef symétrique : AES

- Arithmétique des polynômes binaires

- Un corps binaire $\mathbb{F}_2$ est l'ensemble $\{0; 1\}$ munit des lois d'addition et de multiplication suivante

- Addition

$$(x^5 + x^3 + x^2 + 1) + (x^7 + x^5 + x + 1) = x^7 + x^3 + x^2 + x$$

$$(00101101) \oplus (10100011) = (10001110)$$

$$(2d) \oplus (a3) = (8e)$$

- Multiplication

$$(01110011) \bullet (10100101)$$

$$(x^6 + x^5 + x^4 + x + 1) \bullet (x^7 + x^5 + x^2 + 1)$$

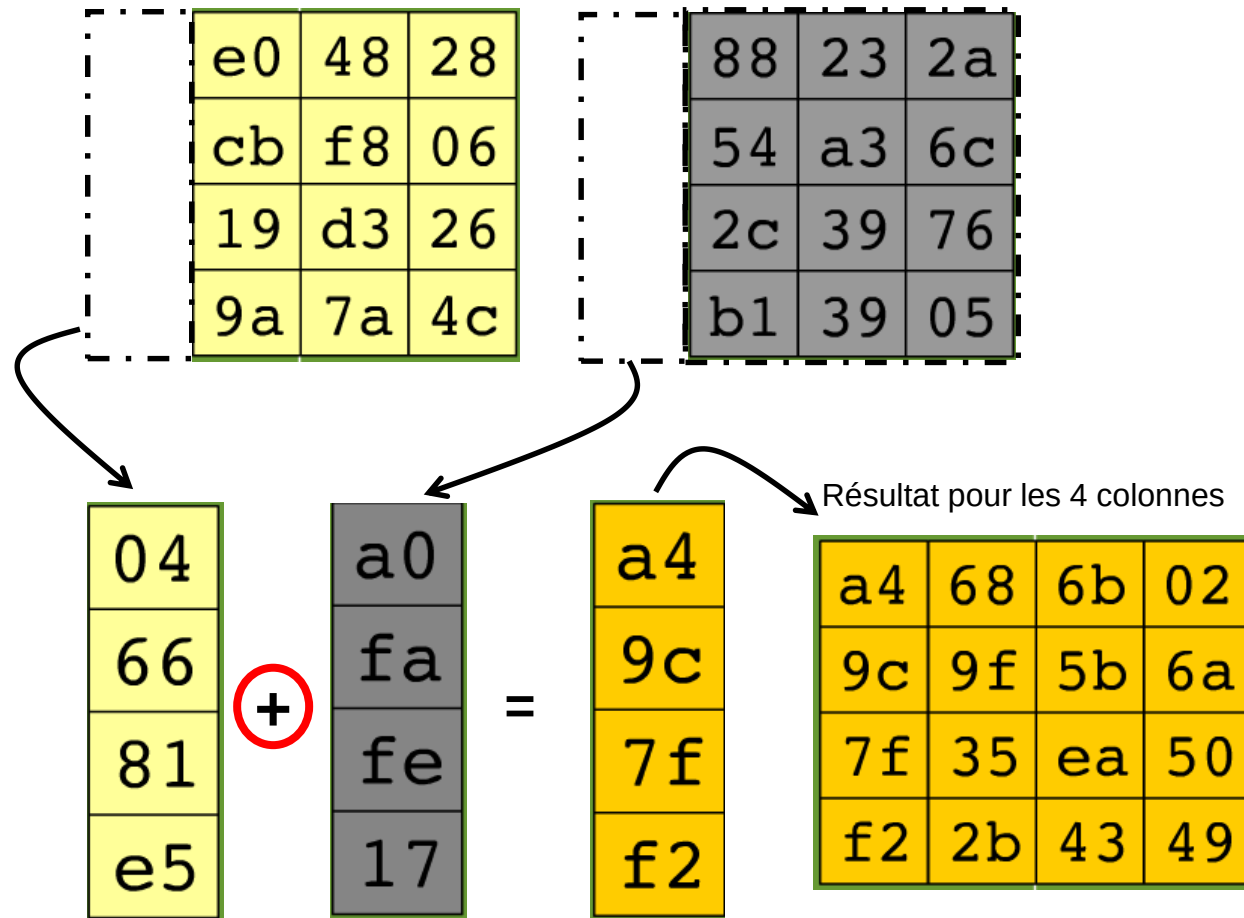
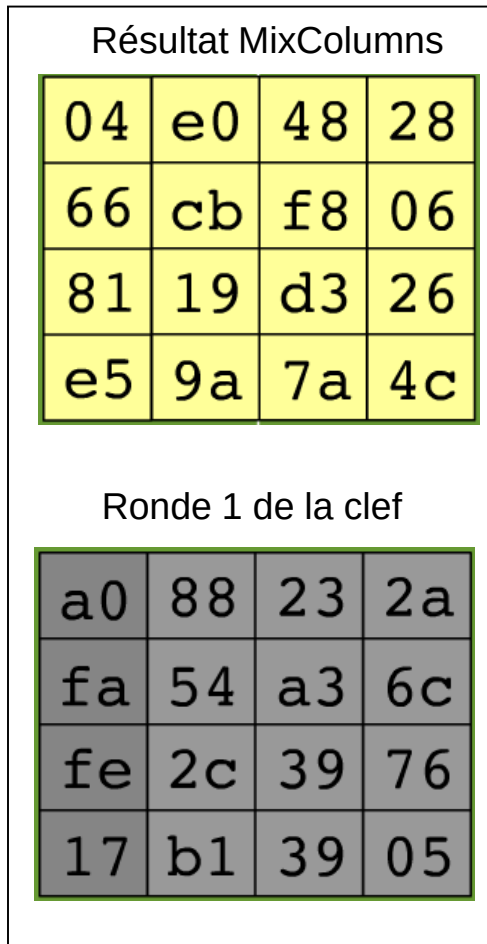
$$= x^{13} + x^{12} + x^{10} + x^9 + x^6 + x^4 + x^3 + x^2 + x + 1$$

- Un polynôme binaire $A(X)$ est une somme formelle en une indéterminée X avec des coefficients dans $\mathbb{F}_2$

$$A(X) = a_0 + a_1X + a_2X^2 + \dots + a_nX^n$$

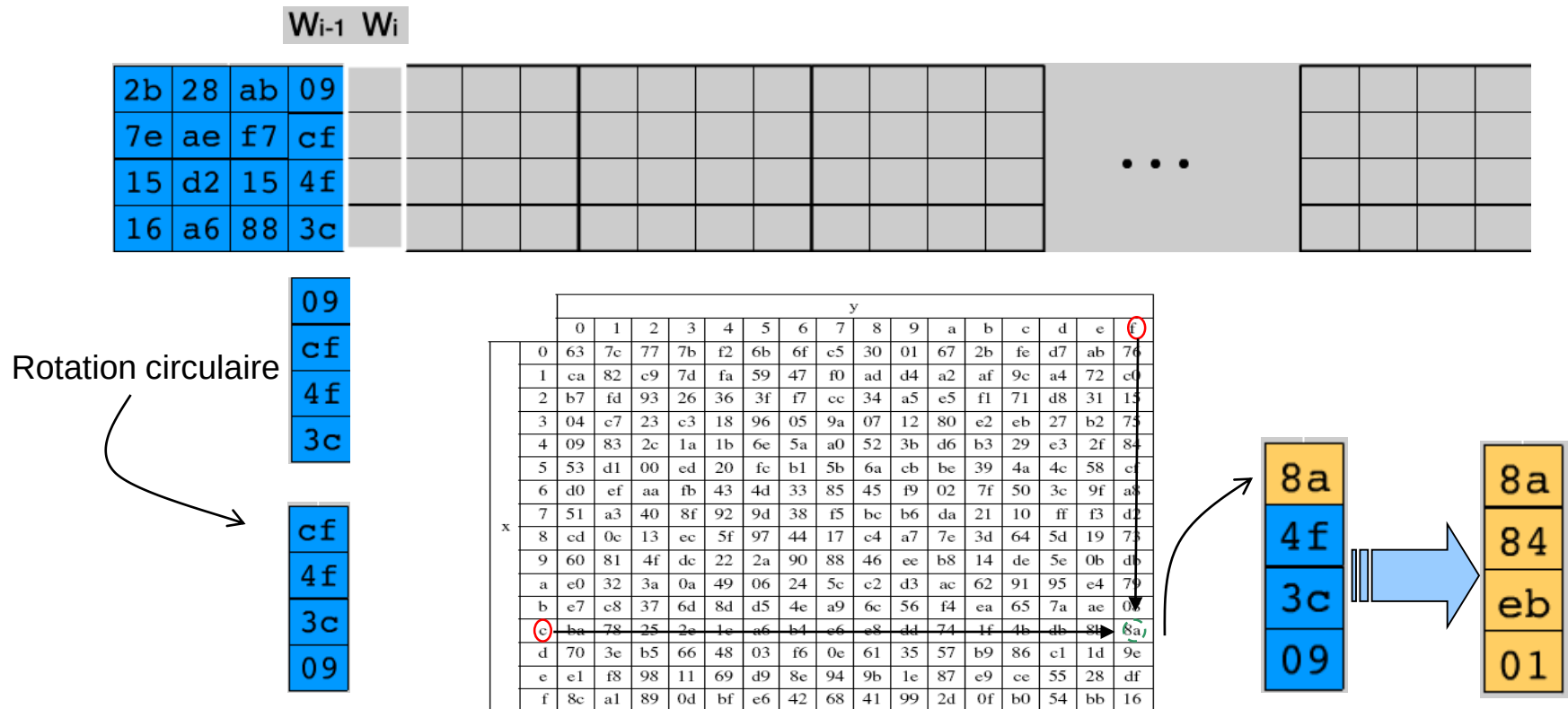
Chiffrement à clef symétrique : AES

- AddRoundKey (State, Ki)

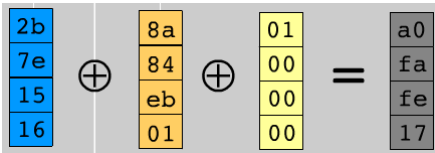
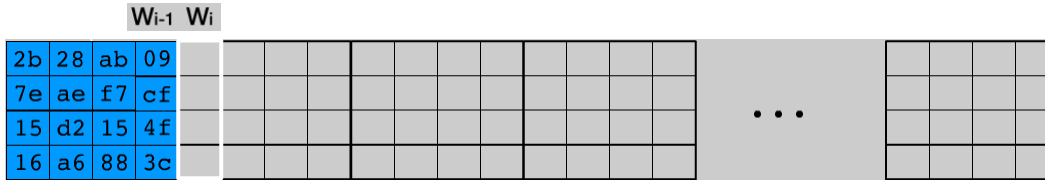


Chiffrement à clef symétrique : AES

- Diversification de la clef (*KeyExpansion*)
 - Permet de diversifier la clef K de $4 \cdot N_k$ dans une clef étendue W de $4N_b(N_r+1)$ octets

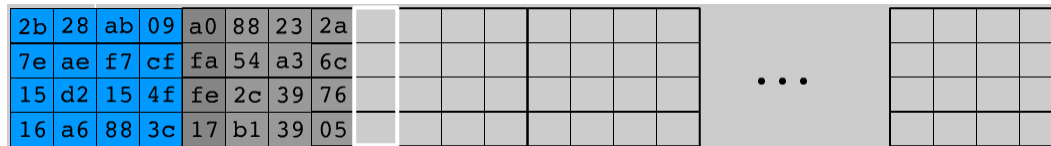
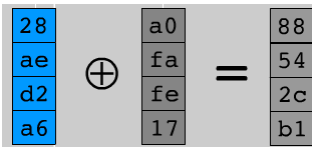
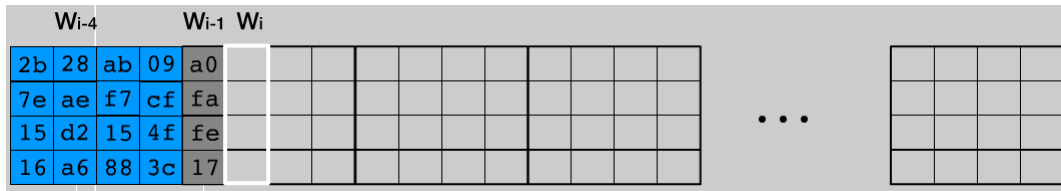


Chiffrement à clef symétrique : AES



01	02	04	08	10	20	40	80	1b	36
00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00

Tableau de constante de tours Rcon[i]



Rotation

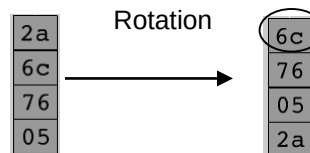


Tableau Sbox

50
38
6b
e5

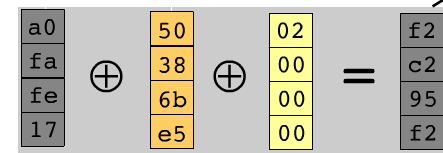
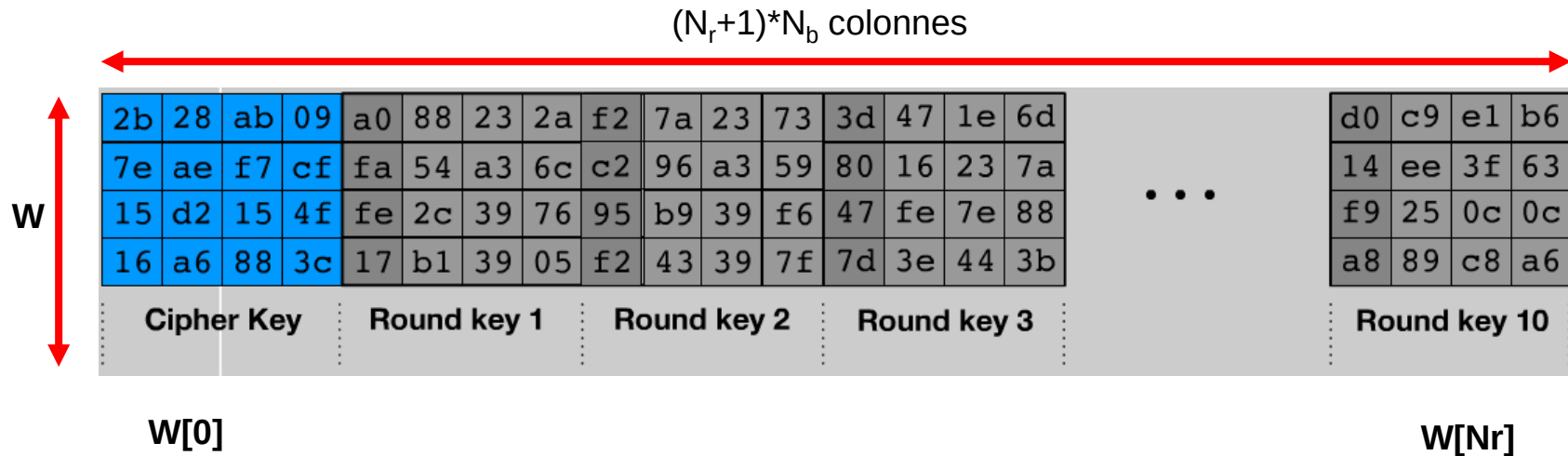


Tableau Rcon[i]

Chiffrement à clef symétrique : AES

- Clef étendue de 10 tours



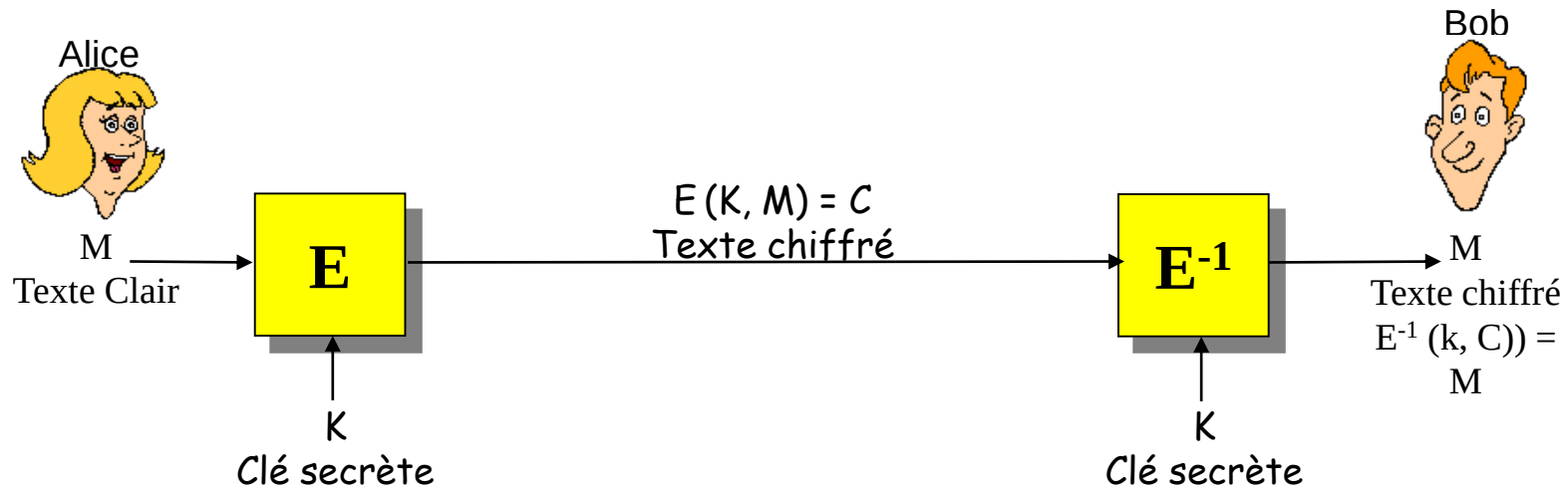
Chiffrement à clef symétrique : AES

- Avantages et spécificités
 - Grande résistance à toutes les attaques connues
 - Très grande rapidité (chiffrement/déchiffrement)
 - Véritable structure mathématique
 - Clé de 256 bits largement suffisant pour les applications commerciales
 - 80 % des acteurs du marché utilisent AES
- Attaques ?
 - Pour l'instant, AES n'a pas été cassé
 - Pas de failles avec une clef de chiffrement 128 bits
 - Principales failles (niveau du logiciel et partage de clés)

Algorithmes de partage de secret/clef

Algorithmes de chiffrement symétrique

Service de confidentialité



- Problème d'échange de clé
 - Le réseau n'est pas de confiance
- Une clé par couple
 - Gestion et renouvellement

Protocole d'échange de clé Diffie Hellman



Alice



Bob

Mise en accord publique sur 2 nombres premiers g et p

Génère une clef secrète a

Génère une clef secrète b

Calcule son élément public

$$A = g^a \bmod p$$

Envoie A à Bob

Calcule son élément public

$$B = g^b \bmod p$$

Envoie B à Alice

$$\text{Calcule } K = (B)^a \bmod P$$

$$\text{Calcule } K = (A)^b \bmod P$$



Attaquant : aucune information sur K à partir de A et B

Protocole d'échange de clé Diffie Hellman



Alice



Bob

$g=11$ et $p=13$

clef secrète $a=5$

$A = g^a \bmod p = 11^5 \bmod 13$

$A = 161051 \bmod 13$

Envoie $A = 7$ à Bob

Calcule $K = (B)^a \bmod p$
alors $K = (2)^5 \bmod 13$

$K = 32 \bmod 13$

$K=6$

clef secrète pour un algorithme
symétrique choisi (chiffrement et
déchiffrement)

$A=7$

clef secrète $b=19$

$B = g^b \bmod p = 11^{19} \bmod 13$

$B = 61159090448414546291 \bmod 13$

Envoie $B=2$ à Alice

Calcule $K = (A)^b \bmod p = (7)^{19} \bmod 13$

$K = 11398895185373143 \bmod 13$

$K=6$

$B=2$



$K = ?$

$(A=7, B=2, g=11, p=13)$

Je ne connais pas a et b !!

DH est sensible à l'attaque MITM !!!

Algorithmes de chiffrement asymétrique

Rappel algorithme RSA

- RSA

- Développé en 1977 par Ron Rivest, Adi Shamir et Len Adleman
- Algorithme asymétrique de cryptographie à clé publique
- Plusieurs services de sécurité

- But

- Transmettre un message codé entre 2 entités
- Seul le récepteur “officiel” puisse le décrypter

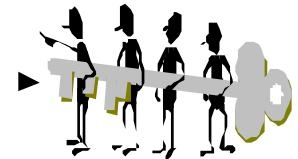
- Principe

- Basé sur la difficulté de factoriser
 - Un nombre très grand quand ce nombre est le produit de deux nombres premiers très grands eux aussi

Rappel algorithme RSA

- M : message à chiffrer (*Plaintext*)
- C : message chiffré (*Ciphertext*)
- n : produit de deux nombres premiers
- Algorithme RSA
 - Clé publique : couple (e, n)
 - Clé privée : d
- Pour chiffrer le message M , on calcule :
 - $C = M^e \text{ modulo } n$
- Pour déchiffrer C on calcule :
 - $M = C^d \text{ modulo } n$

Clef publique (e, n)

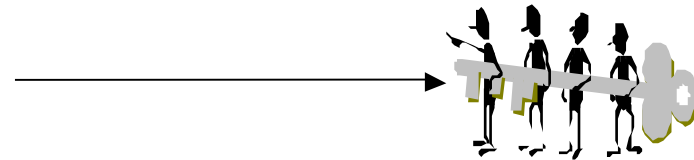
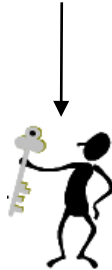


Clef privée d



Rappel algorithme RSA

- M : message à chiffrer (*Plaintext*)
- C : message chiffré (*Ciphertext*)
- Pour chiffrer le message M :
 - On choisit deux grands entiers naturels premiers p et q et on calcule leur produit $n = p \cdot q$
 - On choisit un entier e premier avec $(p-1) \cdot (q-1)$
 - $C = M^e \text{ modulo } n$
- Pour déchiffrer C on calcule :
 - On calcule $e \cdot d \text{ mod } ((p-1)(q-1)) = 1$
 - $M = C^d \text{ modulo } n$
- Clé publique est le couple (e, n)
- Clé privée : d



Rappel algorithme RSA

- Choisir au hasard 2 nombres premiers
 - Ex : $p = 13$ et $q = 17$
- Calculer $n = p.q = 13 \cdot 17 = 221$
- On pose $j = (p-1).(q-1) = 12 \cdot 16 = 192$
- Sélectionne e
 - e et j soient premiers entre eux avec $1 < e < j$
 - « Deux entiers a et b sont premiers entre eux, s'ils n'ont aucun facteur en commun »
 - On choisit $e = 5$
 - Clé publique : (221, 5)
- On calcule d tel que :
 - $e.d = 1 \bmod j \Rightarrow 5.d = 1 \bmod 192$
 - clé privée $d = 77$

Théorème de Bezout

Rappel algorithme RSA

- Chiffrement RSA

- Clé publique pour chiffrer
- Clé privée pour déchiffrer
- Ex: soit $p=13$, $q=17$, $e=5$ et $M=98$
 - $K_{pb}=(192, 5)$ et $K_{pr}=77$
 - $C = M^e \text{ modulo } n \Rightarrow C=115$
 - $M = C^d \text{ modulo } n \Rightarrow M=98$

- Signature

- Clé publique pour vérifier
- Clé privée pour signer (chiffrer !)
- Ex :
 - $C = M^d \text{ modulo } n = 98^{77} \text{ mod } 221 = 115$
 - $M = C^e \text{ modulo } n = 115^5 \text{ mod } 221 = 98$

Rappel algorithme RSA

- Exercice



Alice

$p=13, q=17$
 $K_A^+ = ?$
 $K_A^- = ?$



BoB

$p=3, q=11$
 $K_B^+ = ?$
 $K_B^- = ?$

$C = ?, S = ?$

$M = ?, V = ?$

Message à envoyer $M = \text{« a »}$,
97 en ASCII

Calculer K_A^+ et K_A^- d'Alice.
Calculer K_B^+ et K_B^- de BoB.
Calculer le cryptogramme et la valeur de la signature ?
Comment Bob déchiffre C et vérifie S ?

Rappel algorithme RSA

- Exercice

- Bob et Bernard ont pour clé publique RSA respectivement (n, e_1) et (n, e_2)
- e_1 et e_2 premiers entre eux
- Alice envoie le même message m crypté par les clés publiques RSA de Bob et Bernard
- Eve intercepte les deux messages cryptés et trouve m
- Application numérique : $m=2, n=21, e_1=5, e_2=13$

- Solution

- $C_1 = 2^5 \bmod 21 = 11, C_2 = 2^{13} \bmod 21 = 8, e_1.u + e_2.v = 1, u = -5$ et $v = 2$
- $C_1^u . C_2^v \bmod n = m^{e_1.u} . m^{e_2.v} \bmod n = m^{e_1.u + e_2.v} \bmod n = m$
- $C_1^u . C_2^v \bmod n = (2^5)^{-5} . (2^{13})^2 \bmod 21 = 2^1 \bmod 21 = 2$

Rappel algorithme RSA

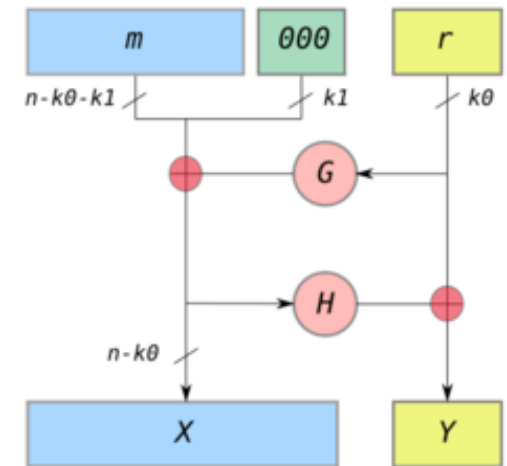
- RSA est un système qui peut être considéré sûr si le message à chiffrer est aléatoire dans l'intervalle $1 \dots N-1$.
- Quand le message consiste en une clé symétrique (AES de 128 bits, 3DES de 168 bits)
 - Ajouter des «0» au bout de la clé n'est pas sûr!
 - Il faut allonger les messages et les rendre aussi aléatoires que possible...
OAEP: Remplissage optimal pour chiffrement asymétrique

Rappel algorithme RSA

- Pour éviter toutes les attaques élémentaires sur RSA :
 - on utilise un padding, ou encodage, du message avant chiffrement
 - En déchiffrement, on vérifie si le padding est correct avant de renvoyer le clair
- Normes :
 - PKCS #1 versions 1.5 et 2.1 proposées par RSA Laboratories en 1993 et préconisent l'emploi d'un padding

Rappel algorithme RSA

- Optimal Asymmetric Encryption Padding (OAEP)
 - Remplissage optimal pour chiffrement asymétrique
 - Standard permettant de chiffrer des messages beaucoup plus courts que ce que la taille de N (dans RSA)
 - $G(r)$: générateur pseudo-aléatoire
 - $H(x)$: fonction de hachage (SHA256)
 - Encodage OAEP d'un message m
 - $X = m \oplus G(r)$
 - $Y = H(X) \oplus r$
 - $OAEP = X || Y$



Rappel algorithme RSA

- Encodage avant le chiffrement par RSA
 - Convertir le message M en code ASCII
 - Découper M en blocs de même longueur $L < n$
 - Chiffrer les blocs un par un
 - Exemple : 'Hello world!'

Index	Character	ASCII Number	Multiplied By	Number
0	H	72	$\times 256^0$	= 72
1	e	101	$\times 256^1$	= 25,856
2	l	108	$\times 256^2$	= 7,077,888
3	l	108	$\times 256^3$	= 1,811,939,328
4	o	111	$\times 256^4$	= 476,741,369,856
5	(space)	32	$\times 256^5$	= 35,184,372,088,832
6	w	119	$\times 256^6$	= 33,495,522,228,568,064
7	o	111	$\times 256^7$	= 7,998,392,938,210,000,896
8	r	114	$\times 256^8$	= 2,102,928,824,402,888,884,224
9	l	108	$\times 256^9$	= 510,015,580,149,921,683,079,168
10	d	100	$\times 256^{10}$	= 120,892,581,961,462,917,470,617,600
11	!	33	$\times 256^{11}$	=
				10,213,005,324,104,387,267,917,774,848
SUM:				10,334,410,032,606,748,633,331,426,632

Rappel algorithme RSA

- Encodage avant le chiffrement par RSA
 - Exemples
 - String 'Hello world!' représente le block 10,334,410,032,606,748,633,331,426,632.
 - String '42!' représente le block 2,175,540
 - 4 = 52 en ASCII
 - 2 = 50 en ASCII
 - ! = 33 en ASCII
 - '42!' = $52 + 256 \cdot 50 + 256^2 \cdot 33$
 $= 52 + 12800 + 2162688$
 $= 2,175,540$
- Chiffrement de '42!' par RSA (3896051,5)
 - $N = p \cdot q = 1999 \cdot 1949 = 3896051$ et $e = 5$
 - $C = M^e \bmod n = 2175540^5 \bmod 3896051 = 3382663$

Rappel algorithme RSA

- Encodage avant le chiffrement par RSA
 - Clé de taille 1024 bits
 - Taille maximale d'un block = 128 octets
 - Message plus long que 128 caractères, on peut séparer les blocs par des virgules

Rappel algorithme RSA

- Pour percer RSA, il faut pouvoir factoriser n .
 - Si on factorise n , on obtient p et q ,
 - on calcule $j = (p - 1)(q - 1)$
 - on calcule d , $e.d = 1 \bmod j$
 - La factorisation de grands nombres est complexe
- Deux difficultés pour implémenter RSA:
- La génération de grands nombres premiers (p et q)
 - L'exponentiation avec de grand facteur.
 - Un standard est défini: PKCS 1 (Public Key Cipher System).

Rappel algorithme RSA

- Pour briser RSA, il faut calculer l'exposant de déchiffrement

- $d = e^{-1} \bmod (p-1)(q-1)$ où $pq = n$
- Mais, il faut factoriser n !!!
- Très difficile
- Exemple !!!

- $N =$

310741824049004372135075003588856793003734602284272754572016194882320644051808150455634682967172328678
2437916272838033415471073108501919548529007337724822783525742386454014691736602477652346609

- $P =$

16347336458092538484431338838650908598417836700330 92312181110852389333100104508151212118167511579

- $Q =$

1900871281664822113126851573935413975471896789968515493666638539088027103802104498957191261465571

Rappel algorithme RSA

- Complexité de la factorisation
 - Avec la factorisation classique

$$O\left(\exp\left(\sqrt[3]{\frac{64}{9}n(\log n)^2}\right)\right)$$

- Avec l'algorithme de Shor

$$O(n^2 \log n \log \log n)$$

n est le nombre de bits représentant le produit des nombres premiers

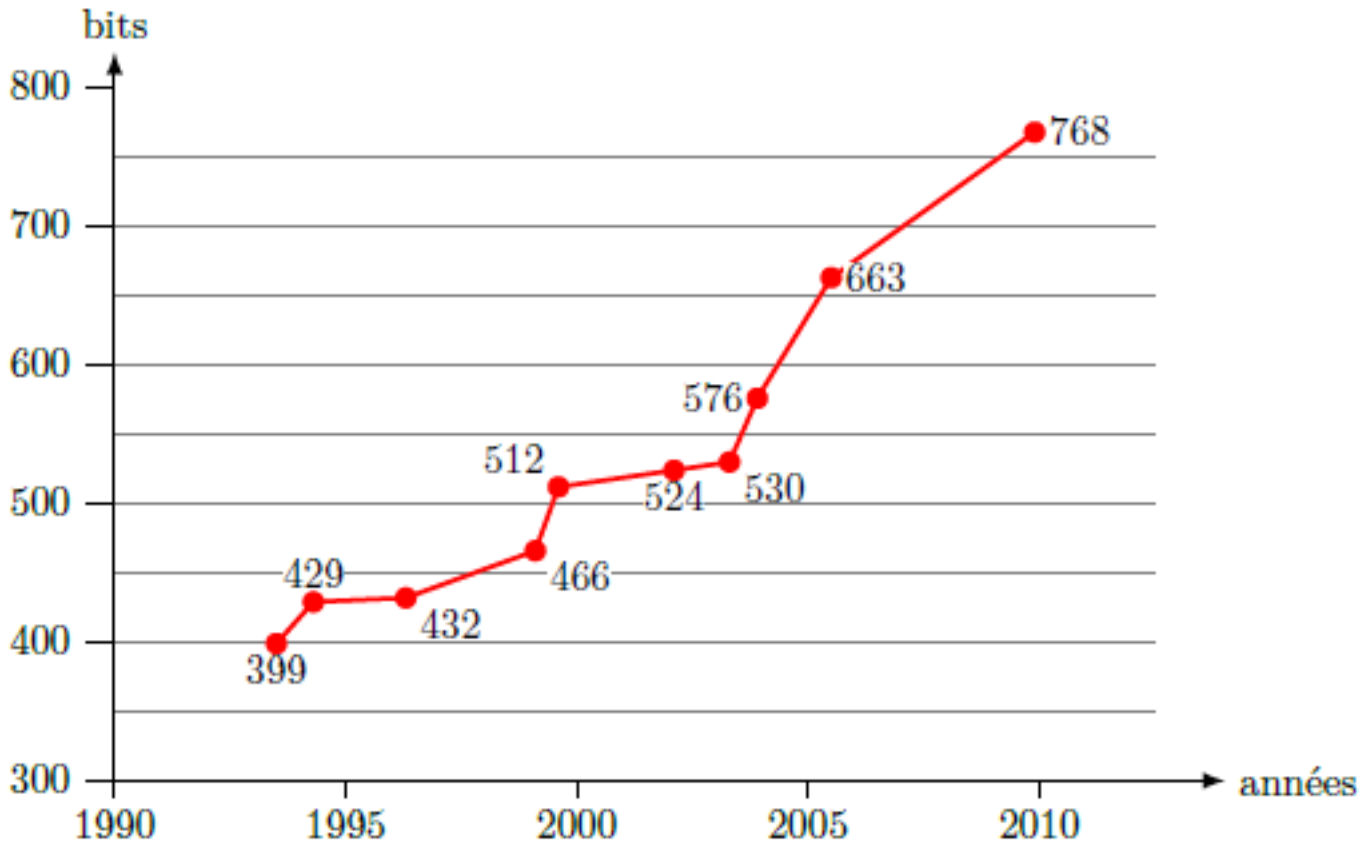
Records de calcul en factorisation

- Records de calculs de factorisation
 - 9 juin 1993
 - Denny, Dodson, Lenstra et Manasse, 397 bits (120 chiffres décimaux) ;
 - 2 avril 1994
 - Atkins Graff, Lenstra et Leyland, 426 bits (129 chiffres décimaux) ;
 - 10 avril 1996
 - Lenstra et al., 432 bits (130 chiffres décimaux) ;
 - 2 février 1999
 - te Riele et al., 466 bits (140 chiffres décimaux) ;
 - 22 août 1999
 - te Riele et al., 512 bits (155 chiffres décimaux) ;
 - 18 janvier 2002
 - Bahr, Franke et Kleinjung, 524 bits (158 chiffres décimaux) ;
 - 1er avril 2003
 - Bahr, Franke, Kleinjung, Lochter et Boehm, 530 bits (160 chiffres décimaux) ;
 - 3 décembre 2003
 - Franke et Kleinjung, 576 bits (174 chiffres décimaux) ;
 - 9 mai 2005
 - Bahr, Boehm, Franke et Kleinjung, 663 bits (200 chiffres décimaux) ;
 - 12 décembre 2009
 - Kleinjung et al., 768 bits (232 chiffres décimaux).

Source: ANSSI, Référentiel Général de Sécurité, Mécanismes cryptographiques, Règles et recommandations concernant le choix et le dimensionnement des mécanismes Cryptographiques, 2014

Records de calcul en factorisation

- Records de factorisation



Source: ANSSI, Référentiel Général de Sécurité, Mécanismes cryptographiques, Règles et recommandations concernant le choix et le dimensionnement des mécanismes Cryptographiques, 2014

Rappel : Algorithme de Shor

- Peter Shor trouve en 1985 un algorithme pour factoriser un nombre en temps polynomial
- Algorithme
 - Prendre un nombre pseudo-aléatoire $a < N$
 - Si a n'est pas premier avec N :
 - on a un diviseur de N
 - Si a est premier avec N ,
 - Rechercher la période r de $f(x) = a^x \bmod N$ tel que $f(x+r) = f(x)$
 - Si r est impair, prendre un nouveau aléatoire a
 - Si on connaît r , alors on a une chance sur 2 environ que r soit pair et alors les facteurs de N sont $\text{PGCD}(a^{r/2} \pm 1, N)$



Rappel : Algorithme de Shor

- Algorithme de Shor
 - Soit $N=21$, on choisit $a=2$

$$2^0 \pmod{21} \equiv 1 \pmod{21}$$

$$2^1 \pmod{21} \equiv 2 \pmod{21}$$

$$2^2 \pmod{21} \equiv 4 \pmod{21}$$

$$2^3 \pmod{21} \equiv 8 \pmod{21}$$

$$2^4 \pmod{21} \equiv 16 \pmod{21}$$

$$2^5 \pmod{21} \equiv 11 \pmod{21}$$

$$2^6 \pmod{21} \equiv 1 \pmod{21}$$

$$2^7 \pmod{21} \equiv 2 \pmod{21}$$

$r=6$

repeating

$$2^6 \equiv 1 \pmod{21}$$

$$(2^3)^2 \equiv 1 \pmod{21}$$

8

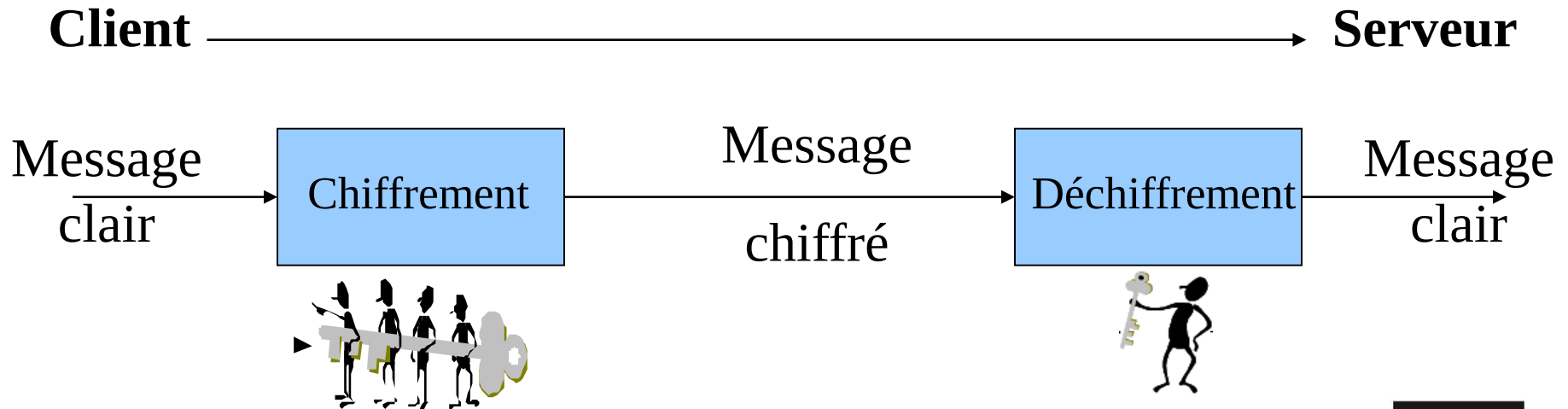
$$\gcd(21, 8 + 1) = \gcd(21, 9) = 3$$
$$\gcd(21, 8 - 1) = \gcd(21, 7) = 7$$

$7 \times 3 = 21$

Rappel algorithme El Gamal

- **Algorithme ElGamal**

- Publié par Tahar El Gamal en 1987
- Sa sécurité dépend de la difficulté de calculer les logarithmes discrets ($3^k \equiv 5 \pmod{7} \Rightarrow K=?$)
- Utilisé pour le chiffrement et la signature électronique.
- Utilisé par le logiciel libre GNU Privacy Guard, et PGP (Pretty Good Privacy)



Paramètres :

- p , un grand nombre premier
- g , un générateur de Z_p^*
- $a \in Z_{p-1}$, $y = g^a \bmod p$
- Clé Publique: (p, g, y)
- Clé privée : a



Rappel algorithme El Gamal

- Chiffrement :

- On génère un secret aléatoire $k \in \mathbb{Z}_{p-1}$
- Pour chiffrer le message m (avec $m < p$)
 - On calcule $E(m, k) = (r, s)$ tel que

$$r = g^k \bmod p$$

$$s = m \cdot y^k \bmod p$$

- Déchiffrement

- Pour déchiffrer (r, s) , on utilise la clé privée a
- $D(r, s) = r^{p-a-1} \cdot s \bmod p = m \cdot g^{ak} g^{-ak} \bmod p = m$.

$$\begin{aligned} - r^{p-a-1} \cdot s &= (g^k)^{p-a-1} \cdot m(g^a)^k \\ &= m \cdot [(g^{p-1})^k (g^k)^{-a}] \cdot (g^k)^a \\ &= m \cdot 1^k \cdot (g^k)^{-a} (g^k)^a \\ &= m \cdot 1, \quad \text{avec } (g^k)^{-a} (g^k)^a = 1 \\ &= m. \end{aligned}$$

$g^{p-1} = 1 \bmod p$ (p :prime)
petit théorème de Fermat

Rappel algorithme El Gamal

- Application numérique (Encryption/Decryption)
 - Soit $g=2$, $p=17$ et $a=5$
 - La clé publique est : $y=g^a \bmod(p)=2^5 \bmod 17=15$
 - La clé privée est $a=5$
 - Chiffrement d'un message $m=9$
 - On génère un nombre aléatoire $k=7$
 - On calcule r et s
$$r = g^k \bmod p = 2^7 \bmod 17 = 9$$
$$s = m.y^k \bmod p = 9 \cdot 15^7 \bmod 17 = 4$$
 - Déchiffrement du $(r=9, s=4)$
 - $D(r, s) = r^{p-a-1} \cdot s \bmod p = 9^{17-5-1} \cdot 4 \bmod 17 = 9 \Rightarrow m$

Rappel algorithme El Gamal

- Signature

- On génère un k tel que $\text{PGCD}(k, p-1)=1$
- On calcule
 - $r=g^k \bmod p$
 - $s=k^{-1} \cdot (m-a \cdot r) \bmod (p-1)$
- La signature de m est donc: (m, r, s)

- Vérification

- On calcule
 - $v1 = y^r \cdot r^s \bmod p$
 - $v2 = g^m \bmod p$
- Si $v1=v2$ la signature est donc valide

Rappel algorithme El Gamal

- Application numérique (Signature/Verification)

- Soit $m=15$, $a=16$, $g=7$ et $p=71$

- Signature

- On génère $k=31$

- On calcule

- $r=g^k \bmod p = 7^{31} \bmod 71 = 11$

- $s=k^{-1} \cdot (m-a \cdot r) \bmod (p-1) = 31^{-1} \cdot (15-16 \cdot 11) \bmod 70$

- $\Rightarrow s=61(15-176) \bmod 70 = -9821 \bmod 70 = 49$

- La signature est donc $(m=15, r=11, s=49)$

- Vérification

- On sait que $y=g^a \bmod p=19$

- On calcule

- $v1=y^r \cdot r^s \bmod p = 19^{11} \cdot 11^{49} \bmod 71 = 23$

- $v2=g^m \bmod p = 7^{15} \bmod 71 = 23$

- $v1=v2$ la signature est donc valide

$31^{-1} \bmod 70 \Rightarrow 61 \cdot 31 = 1 \bmod 70$
Théorème de Bézout

Cryptosystème de Paillier

- Proposé par Pascal Paillier en 1999
- Génération de clés
 - On choisit p et q deux nombres premiers tels que $\text{pgcd}(p; q-1)=1$ et $\text{pgcd}(p-1; q)=1$
 - On calcule $n=p*q$
 - On choisit g dans $\mathbb{Z}^*_{n^2}$
 - *Clef publique: (n et g)*
 - *Clef privée: ($\lambda(n)$ et μ)*
 - On calcule la fonction de Carmichael $\lambda(n) = \text{ppcm}[(p-1)(q-1)]$
 - Le théorème de Carmichael garantit que $g^{\lambda(n)} \equiv 1 \pmod n$.
 - $L(u) = (u-1)/n$
 - $L(g^{\lambda(n)} \pmod{n^2}) = k$
 - $\mu \equiv k^{-1} \pmod n$



Cryptosystème de Paillier

- Cryptosystème de Paillier
 - On choisit p et q deux nombres premiers tels que $\text{pgcd}(p; q-1) = 1$ et $\text{pgcd}(p-1; q) = 1$
 - On calcule $n = pq$.
 - On choisit un élément $g = 1 + kn$ d'ordre n .
 - On calcule $\lambda = \text{ppcm}(p-1; q-1)$ et
 - $\mu = (\lambda \cdot \text{pgcd}(g^{\lambda}-1; n))^{-1} \bmod n$
 - La clé publique est $(g; n)$ et la clé secrète est $(\lambda; \mu)$.
- Pour chiffrer un message $m \in \mathbb{Z}/n\mathbb{Z}$
 - On choisit un entier $r \in \mathbb{Z}/n\mathbb{Z}$ au hasard
 - On calcule $c = g^m r^n \bmod n^2$
- Pour déchiffrer le cipher
 - On calcule $m = \mu \cdot \frac{c^\lambda - 1}{n} \bmod n$

Cryptosystème de Paillier

- Génération de clés

- Soit $p = 7$ et $q = 11$, alors $n = p \cdot q = 7 \cdot 11 = 77$
- On sélectionne g dans $\mathbb{Z}^*_{n^2}$, $g = 5652$
- La clé publique est donc : $(n, g) = (77, 5652)$

- Chiffrement :

- Pour chiffrer m , il faut $m \in \mathbb{Z}_n$
- On choisit un entier non-zéro, $r \in \mathbb{Z}^*_n$
- On calcule $c \equiv g^m \cdot r^n \pmod{n^2}$
- Soit $m=42$ et on choisit $r=23$
- $c \equiv (5652)^{42} \cdot (23)^{77} \pmod{5929} \equiv (4019) \cdot (606) \equiv 4624 \pmod{5929}$

Cryptosystème de Paillier

- Pour déchiffrer, $m \equiv L(c^{\lambda(n)} \bmod n^2) \cdot \mu \bmod n$
 - Il faut d'abord calculer $\lambda(n)$ et μ
 - On calcule la fonction de Carmichael $\lambda(n) = \text{ppcm}[(p-1)(q-1)]$
 - Le théorème de Carmichael garantit que $g^{\lambda(n)} \equiv 1 \bmod n$.
 - $\lambda(77) = \text{ppcm}(6, 10) = 30$
 - On définit $L(u) = (u-1)/n$
 - On calcule $L(g^{\lambda(n)} \bmod n^2) = k$
 - $L(5652^{30} \bmod 5929) = L(3928)$
 - $L(3928) = (3928-1)/77 = 3927/77 = 51$
 - On calcule $\mu \equiv k^{-1} \bmod n$
 - $\mu \equiv 51^{-1} \equiv 74 \bmod 77$
- Déchiffrement
 - Pour déchiffrer $m \equiv L(c^{\lambda(n)} \bmod n^2) \cdot \mu \bmod n$
 - $m \equiv L(4624^{30} \equiv 4852 \bmod 5929) \cdot 74 \bmod 77$
 - $m \equiv 63 \cdot 74 \equiv 4662 \equiv 42 \bmod 77$

Chiffrement de Rabin

- Proposé par Michael Rabin en 1979
- Basé sur la difficulté du problème de la factorisation
- Génération de clefs
 - On choisit deux nombres premiers p et q , tels que $p \bmod 4 = 3$ et $q \bmod 4 = 3$.
 - La clef publique est $n = p \times q$
 - La clef privée est (p, q)



Chiffrement de Rabin

- Chiffrement

- Pour chiffrer m , il faut $m \in \mathbb{Z}_n$ ($m < n$)
- On calcule $C = m^2 \bmod n$

- Déchiffrement

- On calcule tout d'abord m_p , m_q , y_p et y_q tels que
$$m_p = C^{(p+1)/4} \bmod p$$
$$m_q = C^{(q+1)/4} \bmod q$$
$$p \times y_p + q \times y_q = 1$$
- On calcule r et s , tels que
$$r = (p \times m_q \times y_p + q \times m_p \times y_q) \bmod n, \text{ et}$$
$$s = (p \times m_q \times y_p - q \times m_p \times y_q) \bmod n.$$
- L'une des solutions $\{r, s, n-r, n-s\}$ est le message m .

Chiffrement de Rabin

- Application numérique

- Génération de clefs

- Soit $p = 11$, $q = 23 \Rightarrow n = p.q = 253$

- Chiffrement

- Soit $m = 158 \Rightarrow C = m^2 \bmod n = 158^2 \bmod 253 = 170$.

- Déchiffrement

- $m_p = 170^{(11+1)/4} \bmod 11 = 4$.
- $m_q = 170^{(23+1)/4} \bmod 23 = 3$.
- $11y_p + 23y_q = 1 \Rightarrow y_p = -2$ et $y_q = 1$.
- $r = (11 \times 3 \times (-2) + 23 \times 4 \times 1) \bmod 253 = 26$.
- $s = (11 \times 3 \times (-2) - 23 \times 4 \times 1) \bmod 253 = 95$.

- Les solutions possibles sont $\{r, s, n-r, n-s\}$:

- $\{26, 95, (253 - 26) = 227, (253 - 95) = 158\}$.

Fonctions de hachage

Fonction de hachage

- **Caractéristiques**

- Troisième grande famille d'algorithmes utilisés en cryptographie
- Message de longueur quelconque transformé en un message de longueur fixe inférieure à celle de départ
- Message réduit porte le nom de "Haché" ou de "Condensé«

- **Usage**

- Condensé utilisé comme empreinte digitale du message original
- Assure l'intégrité de données

- **Fonctions les plus connues**

- MD5 , SHA-1, SHA-256, SHA512, Whirlpool

* Non prédictibles : les entrées et les sorties sont indépendantes

** considérée cassée lorsqu'il existe un algorithme permettant de trouver une collision

Fonction de hachage

- Caractéristiques

- Fonction unidirectionnelle

- A partir de $H(M)$ il est impossible de retrouver M

- Fonction sans collisions

- A partir de $H(M)$ et M il est impossible de trouver $M' \neq M$ tel que $H(M') = H(M)$

- Exemples

- Fonction de hachage MD5 : empreinte numérique de 128 bits

- $\text{md5}(\text{"Bonjour"}) = \text{ebc58ab2cb4848d04ec23d83f7ddf985}$

- $\text{md5}(\text{"Gendarmerie"}) = \text{5d4a85c5186d9883d453fa7ffec63f2a}$

- $\text{md5}(\text{"La gendarmerie"}) = \text{a1e53f21f46f8f9e889f0712196e9b48}$

Fonction de hachage MD5

- Proposé par *Ronald Rivest* à MIT en 1991, RFC 1321
- Entrée : Message $L \times 512$ bits, sortie : empreinte de taille 128 bits
- Vue d'ensemble
 - Traiter le message par blocs de 512 bits
 - 4 rondes de 16 opérations fonction du bloc (512), des buffers (A, B, C, D) et de fonctions primitives
 - Résultat (4×32 bits) de chaque ronde est utilisé pour l'initialisation des registres suivants
 - Résultat final est obtenu en additionnant A,B,C,D
- Buffers Initialisés à
 - A = 01 23 45 67
 - B = 89 ab cd ef
 - C = fe dc ba 98
 - D = 76 54 32 10
- 4 fonctions non linéaires

$$F(X, Y, Z) = (X \cdot Y) + (\bar{X} \cdot Z)$$

$$G(X, Y, Z) = (X \cdot Z) + (Y \cdot \bar{Z})$$

$$H(X, Y, Z) = X \oplus Y \oplus Z$$

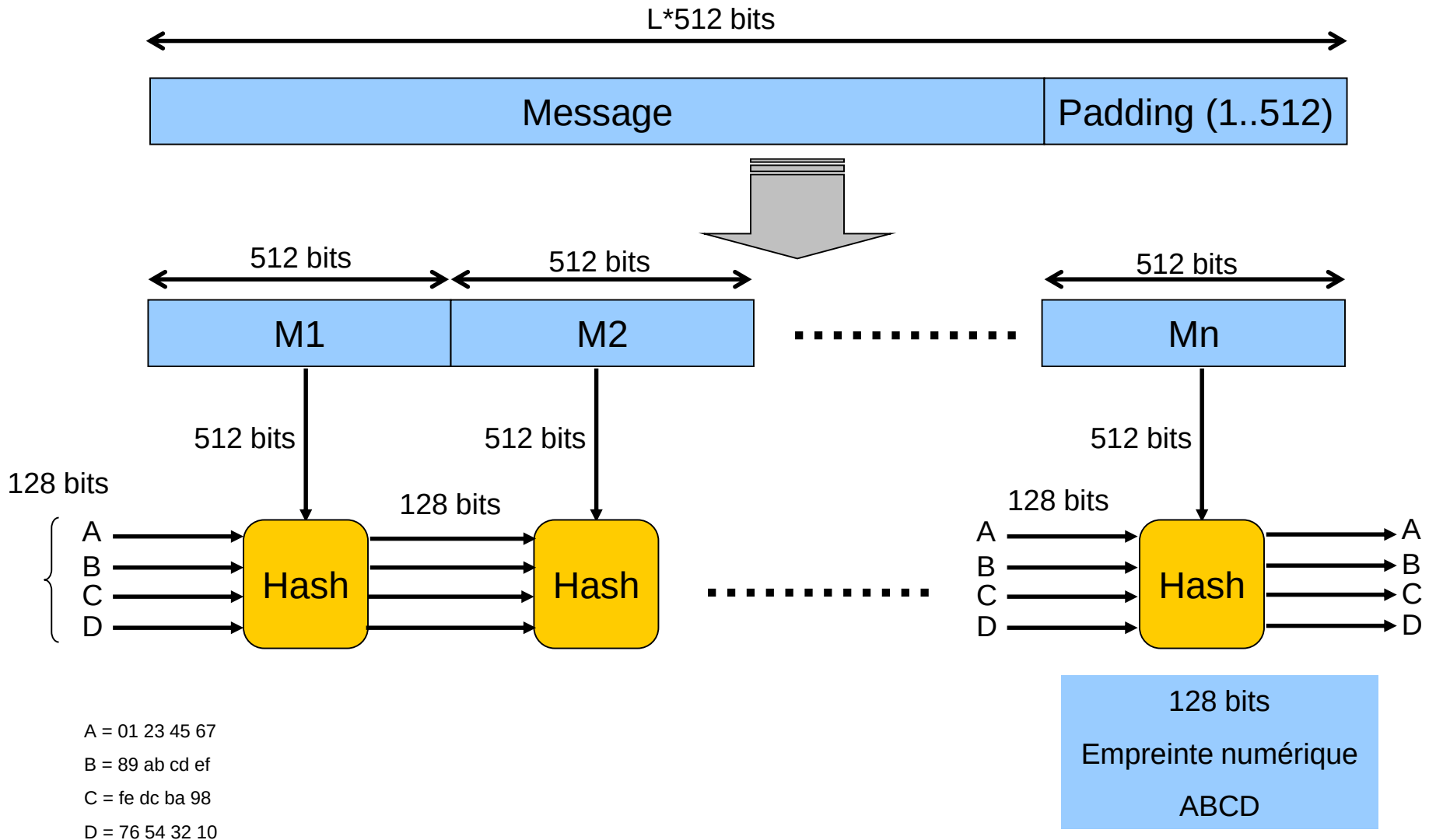
$$I(X, Y, Z) = Y \oplus (X + \bar{Z})$$

Entrée : 3 variables de 32 bits chacune

Sortie : 32 bits

• AND + OR $\oplus$ XOR

Fonction de hachage MD5



Fonction de hachage MD5

- Transformations FF, GG, HH, II

- FF(a, b, c, d, M[k], s, i) :
$$a = b + ((a + F(b, c, d) + M[k] + T[i] \lll s)$$

- GG(a, b, c, d, M[k], s, i) :
$$a = b + ((a + G(b, c, d) + M[k] + T[i] \lll s)$$

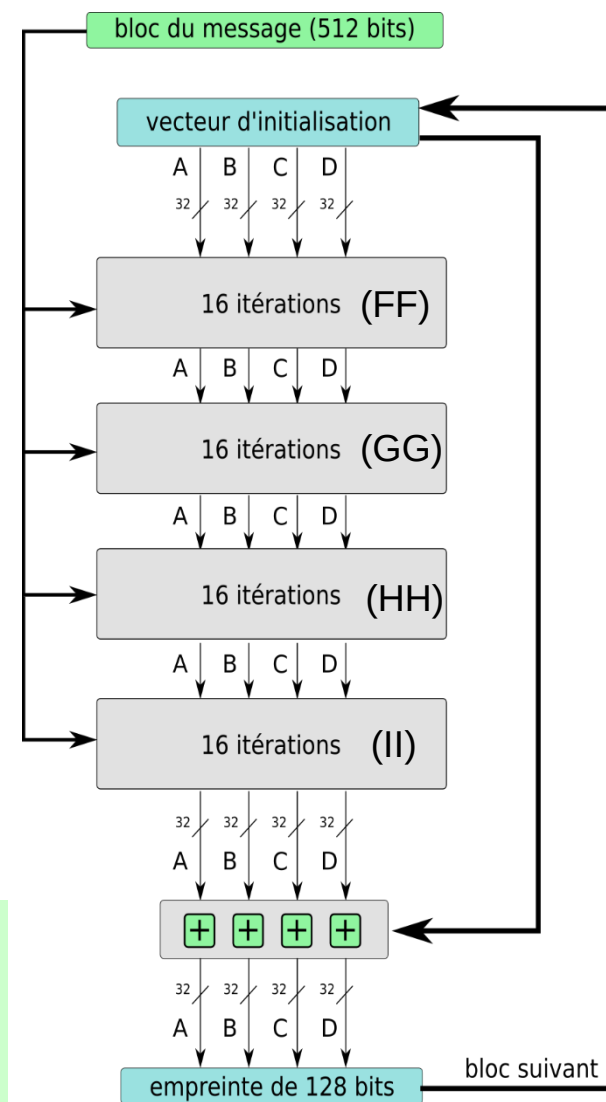
- HH(a, b, c, d, M[k], s, i) :
$$a = b + ((a + H(b, c, d) + M[k] + T[i] \lll s)$$

- II(a, b, c, d, M[k], s, i) :
$$a = b + ((a + I(b, c, d) + M[k] + T[i] \lll s)$$

• T[i] : $i = 1, 2, \dots, 64$, valeur obtenu par la fonction sinus,
 $T[1] = 2^{32} \cdot \text{Abs}(\sin[i])$, avec i en radian. Ex: $T[1] = 4294967296 * 0.8414 =$
d76aa478 en hexa

• s: décalage à gauche de s bits

• M[k] : $0 \leq k \leq 15$, $k^{\text{ième}}$ sous-bloc du message



Fonction de hachage MD5

- Exemple :
 - 152 bits de données : 7a138b25 24af17c3 17b439a1 2f51c5a8 8051cb36
 - Padded message (512bits) = Original message(152bits) + Padding(360bits).
 - A = 67452301, B = efcdab89, C = 98badcfe, D = 10325476
 - Tour 1:
 - $FF[a, b, c, d, M[0], 7, 1]: a = b + ((a + F(b, c, d) + M[k] + T[l] \lll s), a=b+U$
 - $a = 67452301$
 - $b = efcdab89$
 - $M[0]=7a138b25$
 - $T[0]=d76aa478$
 - $F(b,c,d)= b.c + b.d = 98badcfe$
 - Alors $U= efcdab89 + (67452301+98badcfe+ 7a138b25 + d76aa478 \lll 7)$
 - $a=b + U: efcdab89 + bf17ce28 = aee579b1$
 - Il faut faire encore 15 autres itérations de FF pour obtenir A, B, C et D de la première tour
 - Répéter l'opération avec GG, HH et II pour obtenir la signature